
투자 / 증권

IT 관점 핵심 흐름

증권 기초부터 HTS/OMS, 매칭, 야간배치 및 백엔드 아키텍처까지

i 증권이란?

재산적 권리를 담아서 거래가 가능하게 만든 것

예) 친구에게 100만 원을 빌려줬다면 그건 둘 사이의 개인적인 약속.
그 권리를 쪼개서 누구나 사고팔 수 있게 공식적인 시장에 올려놓으면 그게 바로 **채권이라는 증권**입니다.

증권의 핵심 속성 3가지

권리의 표준화

01

내용이 모호하면 거래가 안 됨.

"삼성전자 주식 1주 = 이익 배당받을 권리 + 의결권" 처럼 내용이 명확하게 정의되어야 시장에서 사고팔 수 있음.

양도 가능성

02

그 권리를 남에게 팔 수 있어야 함.

팔 수 없으면 시장 자체가 생기지 않음.

가치의 변동성

03

기업 실적, 금리, 지수 등 기초가 되는 무언가의 가치에 따라 증권 가격이 계속 바뀜.

이 변동성이 있기 때문에 투자의 기회도 리스크도 생기는 것.

증권의 종류

종류	담긴 권리
주식 (Equity)	기업 소유권의 일부
채권 (Bond)	돈을 빌려줬다는 청구권
파생증권	다른 자산 가격에서 파생된 권리
펀드 / ETF	여러 자산을 묶은 지분

증권은 왜 존재하는가?



돈이 필요한 곳과 돈을 불리고 싶은 곳을 연결하기 위해서

- 기업: 주식을 발행해서 투자자에게 팔고 → 자금을 조달 (상장의 이유)
- 투자자: 주식을 사서 → 기업이 성장하면 이익을 얻음

증권시장은 이 직접금융의 핵심 인프라

HTS란?

투자자가 사용하는 증권 거래 프로그램 (클라이언트)

- ✓ 시세 조회, 계좌 조회, 매수·매도 주문 기능을 제공합니다.
- ✓ 주문을 직접 처리하지 않고, 뒷단인 OMS로 전달하는 역할을 수행합니다.

</> IT 관점에서의 역할	
UI 제공	사용자가 주문을 입력할 수 있는 화면 제공
데이터 생성	계좌번호, 종목코드, 가격, 수량, 매수/매도 구분 등
통신	API 및 네트워크(TCP/IP, HTTPS)를 통해 OMS 전송



OMS는 HTS로부터 들어온 주문을 접수하고, 내부 원장(계좌 잔고 등)을 확인한 뒤 실제 거래소로 주문을 전달하는 '주문 중앙 통제 센터'입니다.

비즈니스 역할 (Process Flow)



IT 관점의 핵심 기술 3가지

초저지연 처리

(Low-Latency)

금융 주문은 속도가 생명입니다. 네트워크 지연이나 DB 접근 시간을 최소화하기 위해 디스크 기반 DB(Oracle 등) 대신 **인메모리 데이터베이스(예: Redis)**를 활용하여 호가와 주문 상태를 메모리 위에서 초고속으로 처리합니다.

메시지 큐 & 비동기 아키텍처

장 시작 직후(오전 9시)처럼 트래픽 폭주 시 시스템이 다운되지 않도록, **Kafka나 RabbitMQ** 같은 메시지 브로커를 활용해 주문 요청을 큐(Queue)에 쌓아두고 순차적이면서도 빠르게 병렬 처리합니다.

엄격한 정합성과 분산 트랜잭션

내 계좌에서는 돈이 빠져나갔는데 체결은 안 되는 최악의 사고를 막아야 합니다. 여러 시스템 간의 상태를 100% 일치시키는 강력한 **트랜잭션 관리(2-Phase Commit 등)**가 필요합니다.

FEP란?

증권사와 한국거래소(KRX) 사이에서 주문을 주고받는 **중계 서버**.
투자자는 거래소에 직접 접속할 수 없으므로, FEP가 주문을 받아 거래소에 전달하고 체결 결과를 다시 가져옵니다.

주고받는 정보

- ✓ **주문:** 고객 주문을 거래소로 전달
- ✓ **체결:** 체결 결과를 고객에게 전달
- ✓ **시세:** 실시간 호가·가격을 전달

FEP에서 속도는 체결 우선순위의 핵심 요소

가격이 같을 경우 0.001초라도 느리면 다른 주문에 밀려 체결되지 않을 수 있습니다.

항목	기존	Exture 3.0 (KRX)
처리 속도	70마이크로초	50마이크로초
일일 처리 용량	4억 2천만 건	9억 4천만 건
주문 대역폭	12Mbps	45Mbps

속도가 생명!

💡 IT 관점 핵심 포인트

비동기 처리

응답 대기 중에도 다른 주문 동시 처리

드롭 카피

장애 시에도 체결 내역을 놓치지 않는 백업 채널

호가제출시간 관리

밀리초(1/1000초) 단위 정밀 관리 의무

매칭

정해진 규칙에 따라 매수·매도 주문을 자동으로 짝지어 거래 성사

체결

매칭 성립으로 거래 확정.
체결가·수량·시각 동시 확정 (취소불가)

체결통보

거래소 → 증권사 → 투자자 전달.
유실·지연 불가 핵심 요소

매칭의 핵심 규칙 2가지

가격 우선 (Price First)

1

가격 차이가 있으면 순서가 이미 결정됨. 시간은 무관!

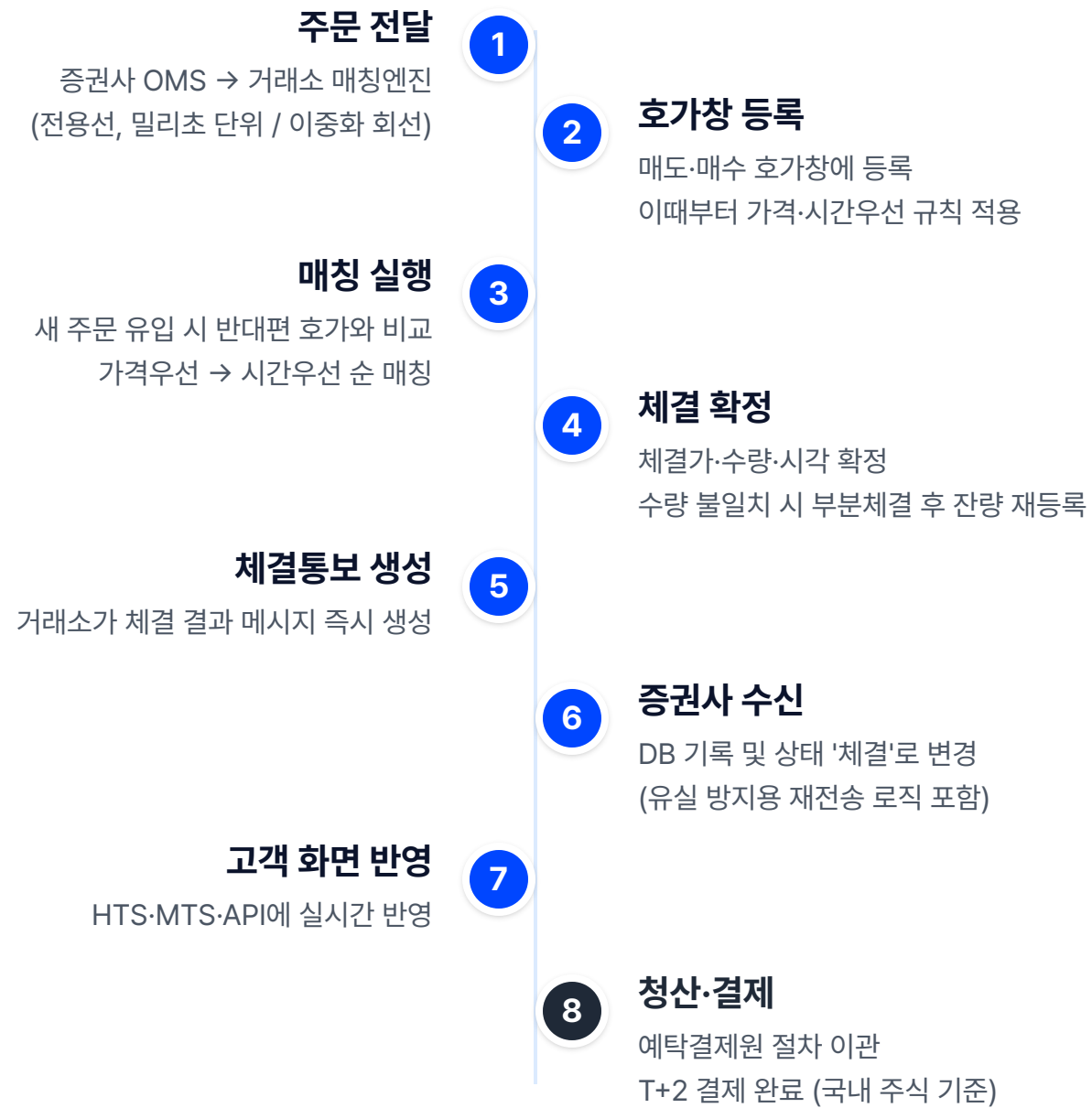
- **매수 주문:** 가장 **높은** 가격을 제시한 주문 우선
- **매도 주문:** 가장 **낮은** 가격을 제시한 주문 우선

시간 우선 (Time First)

2

가격이 동일한 주문이 몰렸을 때 적용되는 "타이브레이커"

- 먼저 접수된 주문이 먼저 체결됨



주문 유입/체결 시 계좌의 예수금과 보유 수량을 즉시 갱신하여 다음 주문 검증에 반영하는 단계.

핵심: 정합성과 동시성 제어



구현 시 핵심 고려 포인트 3가지

동시성 제어

같은 계좌에 여러 주문이 동시에 들어올 때 잔고 갱신이 꼬이지 않게 보호.

낙관적 락

비관적 락

트랜잭션 경계

잔고 차감과 주문 데이터 생성은 반드시 **같은 트랜잭션**에서 처리되어야 무결성이 보장됨.

멍등성 처리

이벤트(응답)가 중복 전달되어도 같은 결과가 두 번 반영되어 잔고가 이중으로 변동되지 않도록 설계.

T 일

● 당일 체결 데이터 집계 & 매매확인

거래소 수신 데이터(FEP)와 내부(OMS) 잔고 정합성 검증. 거래소가 증권사별 순매수/채무 계산 확정.

증권사 야간배치 (EOD Batch) 실행

손익 재계산

예수금(D+2) 갱신

증거금 부족/반대매매 산출

제세금/수수료/이자 정산

원장(Ledger) 확정 반영

T+1 일

● 예탁결제원(KSD) 결제 지시 송신

증권사가 KSD에 누가 어떤 종목을 주고받을지 D+2 기준으로 전송

T+2 일

● 실제 결제 및 원장 확정 (BOD)

KSD가 증권/대금 실제로 이전 (법적 소유권 확정). 결제 완료 데이터를 반영해 익일 서비스(BOD) 오픈.

가장 큰 고민: 정합성과 역등성

짧은 시간 내에 전 고객 손익/잔고 재계산. 중간에 서버가 죽거나 재실행될 때 **잔고가 두 배로 불어나는 사고(중복 반영)**를 막는 것이 핵심.

→ 처리 상태 플래그, 트랜잭션 기반 중복 방지 로직 필수.

❌ 기존 방식: Cron 스크립트

시간 기준 예약 실행.

위험성: 앞 작업(수수료 계산)이 안 끝났거나 에러가 났는데, 시간 됐다고 다음 작업(최종 결제)을 댈다 실행해 대형 사고 유발 가능.

📊 최신 트렌드: DAG 기반 오케스트레이션

DAG (방향성 비순환 그래프)

"A가 끝나면 → B,C 동시 실행 → B,C 둘다 성공해야 → D 실행" 의존성 명확히 정의

ETL 워크플로우

Extract(추출) → Transform(가공/손익계산) → Load(최종 원장 적재)

Apache Airflow 도입 (ex. 카카오뱅크)

복잡한 수천 개의 ETL 작업이 엉키지 않게 통제. 실패 시 다음 작업 정지, 실패 지점부터 재실행 및 개발자 알림 제공.

1 BigDecimal 사용

돈 계산은 절대 부동소수점 오차가 허용되지 않으므로 필수.

2 원장 (Ledger) 설계

모든 변동을 트랜잭션 로그로 남기고, 잔고는 그 로그의 합으로 검증 가능해야 함.

3 명확한 상태머신 (State Machine)

접수 → 대기 → 부분체결 → 완전체결. 잘못된 전이는 시스템 레벨에서 차단.

4 동시성과 정합성 > 성능

속도보다 데이터의 정확성과 무결성이 최우선 원칙.

5 멱등성 (Idempotency)

네트워크 재시도/중복 요청 시에도 동일 주문이 두 번 처리되지 않도록 보장.

6 시간과 순서의 정확성

시간우선/가격우선 원칙 때문에 서버 간 시계 동기화(NTP 등)와 순서 보장 필수.

7 규제와 감사 대응 (Compliance & Audit)

모든 변경 이력이 삭제 불가 형태로 남아야 하고 재현 가능해야 함.

핵심 용어

T+2 결제제도

주식 매매 당일(T)이 아닌 3영업일(T+2)에 실제 주식과 대금이 교환됨. IT 관점에서는 상태가 Matched에서 Settled로 전이되며 Master Ledger로 Commit되는 트랜잭션 과정.

원장 (Ledger)

증권사가 고객 계좌, 거래 내역, 매매 등을 영구적으로 기록하고 관리하는 최상위 마스터 시스템/데이터베이스.

배치 처리 (Batch Processing)

장 마감 후 대량의 데이터를 일괄 처리(BOD/EOD). 스케줄러를 통해 데이터를 읽고 정산/평가 후 DB에 일괄 Write.

출처 및 참고문헌

- IBM, 「배치 작업이란 무엇인가요?」
- 카카오뱅크 기술블로그, Airflow 펀드 배치 시스템
- 금융감독원 파인(FINE) 금융용어사전
- 요즘 IT "데이터 엔지니어링 핵심 도구 'Apache Airflow' 개념 정리"
- 한국거래소 규정/제도 지침 & 뉴스웍스 (한국거래소 공식 발표 기반)